الجمهورية الجزائرية الديمقراطية الشعبية
République Algérienne Démocratique et Populaire

وزارة التعليم العالي والبحث العلمي
المدرسة العليا في علوم و تكنولوجيات
الاعلام الآلي و الرقمنة

Ministère de l'Enseignement Supérieuret de la Recherche Scientifique
École supérieure en Sciences et Technologies de l'Informatique et du Numérique

ESTIN

2ème année CS

Conduite de Projet

Mini-projet « Conduite de projet »

Conception et développement d'agents logiciels intégrés à une plateforme commune

Objectif du TP

Les étudiants seront invités à former des équipes de cinq à six personnes afin de réaliser un projet en groupe, tout en respectant les différentes phases de conduite de projet, appliquées au développement d'UN AGENT LOGICIEL destiné à être intégré dans une plateforme commune. Ce travail sera traité dans son ensemble comme un véritable projet. Les équipes devront être constituées avant la 2e séance du TP (date T1).

Les étudiants doivent choisir entre deux catégories d'agent :

- Agent de cybersécurité (détection, audit, protection du parc informatique d'une entreprise) , voir la liste en section 2.1 ;
- ou agent d'automatisation métier (commercial, marketing, stock, logistique, assistant, etc.) — voir la liste en section 2.2

Dans les deux cas, la démarche de conduite de projet, les livrables attendus (L1 à L8) et les règles de cloisonnement du code sont identiques ; seul change le domaine fonctionnel de l'agent que vous développez. Chaque équipe devra produire tous les livrables du projet pour l'agent qui lui aura été attribué ou qu'elle aura choisi, en respectant le plan de projet élaboré par l'équipe.

Notes importantes :

- Sans exception, le projet d'équipe ne pourra pas être réalisé individuellement.
- Il est de la responsabilité de chaque étudiant(e) de s'intégrer à une équipe avant la deuxième séance de TP.
- Chaque équipe développe un seul agent. Deux équipes ne peuvent pas développer le même agent.
- Le projet global n'est réputé réussi que si votre agent, une fois assemblé au noyau commun, fonctionne réellement, seul et, quand c'est applicable, en collaboration avec les autres agents.

1. Ce que fournit l'enseignant (à ne pas redévelopper)

L'enseignant met à disposition de chaque équipe un noyau de plateforme déjà développé (authentification, gestion des utilisateurs/entreprises, catalogue d'agents, passerelle d'intégration) ainsi que les documents contractuels ci-dessous. Votre travail ne porte pas sur ce noyau : il porte sur VOTRE agent, qui doit s'y connecter en respectant ces documents.

- AGENT_INTEGRATION_CONTRACT.doc — le contrat technique non négociable que votre agent doit respecter pour s'intégrer au noyau (endpoints obligatoires, format des réponses, sécurité).
- INTER_AGENT_DATA_CONTRACT.doc — pour les agents métier uniquement : le registre des objets métier partagés (Commande, Stock, Expédition, Facture...) qu'un agent peut produire ou consommer.
- DEVELOPER_INSTRUCTIONS.doc — le guide pas-à-pas pour développer et soumettre votre agent.

Ces documents sont les SEULES informations techniques communes entre équipes. Ils ne remplacent à aucun moment le code de votre agent, qui reste développé de façon autonome par votre équipe (voir section 4)

2. Choisissez votre agent

2.1 Agent Cybersécurité

Chaque équipe choisit un seul agent parmi la liste suivante

Agent	Fonction	Outils / techno (gratuits)
Scanner de vulnérabilités	Détecte les failles connues sur le parc réseau du client	Nmap, OpenVAS/GVM, Python
Inventaire des actifs IT	Cartographie automatique du parc informatique	osquery, Nmap, Python/Go
Détection d'intrusion (IDS)	Surveille le trafic réseau, alerte sur comportements suspects	Suricata/Snort, règles Emerging Threats
Centralisation de logs (mini-SIEM)	Collecte et corrèle les logs multi-machines	Wazuh, ELK Stack, Docker
Audit de configuration (hardening)	Vérifie la conformité aux bonnes pratiques (CIS)	Lynis, OpenSCAP
Anti-ransomware / intégrité fichiers	Détecte les chiffrements massifs suspects	inotify, ClamAV, watchdog (Python)
Simulation de phishing	Teste la vigilance des employés via campagnes simulées	GoPhish (100% gratuit)
Gestion des correctifs (patch mgmt)	Vérifie et déploie les mises à jour de sécurité	Ansible, osquery
Threat intelligence locale	Agrège des indicateurs de compromission (IOC)	MISP, flux OSINT gratuits
IAM léger (gestion des accès)	Centralise l'authentification et les droits	Keycloak
Conformité réglementaire (RGPD/ISO 27001)	Checklists et rapports d'audit automatisés	Django/Flask + génération PDF
Chatbot de sensibilisation cybersécurité	Répond aux questions des employés, forme en continu	Ollama + modèle open source (Llama/Mistral)

2.2 Agent Automatisation métier

Ou chaque équipe choisit un seul agent parmi la liste suivante

Agent	Fonction	Outils / techno (gratuits)	Produit / Consomme
Commercial / CRM léger	Suivi des prospects, pipeline de vente, relances	Python/Django, PostgreSQL, SuiteCRM	Produit: commande, prospect
Marketing automation	Campagnes emailing et réseaux sociaux automatisées	Mautic, Postfix (SMTP), Python	Produit: prospect / Consomme: commande

Gestion de stock	Suit les niveaux de stock, alerte les ruptures	Python, PostgreSQL, module Odoo Stock	Consomme: commande / Produit: stock_level, mouvement_stock
Logistique / livraison	Optimise les tournées, suit les expéditions	OSRM/GraphHopper, Python	Consomme: commande, stock_level / Produit: expedition
Facturation	Génère les factures, suit les paiements	Python, ReportLab, module Odoo Invoicing	Consomme: commande, expedition / Produit: facture
RH léger	Gestion des congés, plannings simplifiés	OrangeHRM, Python	Aucune dépendance externe requise
Support client / ticketing	Gère les demandes et réclamations clients	Zammad, Django	Consomme: commande, facture / Produit: ticket_support
Assistant virtuel interne	Chatbot de support interne / FAQ employés	Ollama + Llama/Mistral (local, gratuit)	Consomme: plusieurs types selon besoin
Veille concurrentielle	Scrape et synthétise l'information marché	Scrapy, BeautifulSoup, Python	Aucune dépendance requise
Génération de contenu marketing	Rédige posts, fiches produits automatiquement	Ollama local (LLM open source)	Consomme: prospect
Reporting / BI	Tableaux de bord consolidés multi-sources	Metabase, PostgreSQL	Consomme: commande, facture, stock_level
Gestion documentaire	Archive et indexe contrats, factures, documents	Nextcloud (open source)	Consomme: facture
Agent a proposer			
Agent a proposer			

Pour ce type d'agent uniquement : les colonnes "Produit / Consomme" indiquent les types d'objets métier que votre agent devra publier et/ou recevoir via le noyau, tels que définis dans INTER_AGENT_DATA_CONTRACT.doc — voir section 3.

3. Collaboration entre agents (agent automatisation métier)

Un utilisateur peut souscrire à plusieurs agents à la fois. Certains doivent alors collaborer pour réaliser des tâches dépendantes : par exemple, l'agent Logistique a besoin de connaître le niveau de stock avant de programmer une expédition, ou l'agent Facturation a besoin de savoir qu'une commande a été expédiée avant d'émettre une facture.

Cette collaboration ne se fait JAMAIS en connectant directement le code de deux agents entre eux : elle passe exclusivement par le noyau, selon un principe à double lecture :

- Un agent publie un objet métier (ex. une commande) ; le noyau notifie automatiquement tout autre agent actif du même utilisateur qui déclare consommer ce type d'objet.
- Si l'utilisateur n'a pas souscrit à l'agent producteur correspondant, il peut alimenter ou consulter le même flux directement depuis son tableau de bord, les agents qui en dépendent continuent alors de fonctionner normalement.

Dès le livrable L1, identifiez précisément quels types d'objets votre agent va produire et/ou consommer, en vous référant à INTER_AGENT_DATA_CONTRACT.doc. C'est la SEULE information technique que vous pouvez avoir besoin de connaître à propos d'un autre agent, mais pas son code source.

Pour les agents de cybersécurité : les agents fonctionnent de façon indépendante et remontent leurs alertes au noyau via le webhook standard décrit dans AGENT_INTEGRATION_CONTRACT.doc ; il n'y a pas d'échange de données métier entre agents de cybersécurité.

4. Règle fondamentale : cloisonnement du code entre équipes

- Chaque équipe ne doit voir, à aucun moment, le code source des autres équipes.
- Seul l'enseignant a une visibilité sur l'ensemble des dépôts de code.
- Les documents contractuels de la section 1 sont, eux, PUBLICS entre toutes les équipes des deux types d'agents concernés : c'est précisément leur rôle de permettre l'intégration sans partage de code.
- Toute équipe qui copierait le code d'une autre équipe s'expose à une sanction pour manquement à l'intégrité académique.

Cette contrainte n'est pas une gêne : c'est la contrainte réelle d'un environnement multi-équipes en entreprise, où seul le contrat d'intégration (et non le code interne d'autrui) garantit que les modules fonctionnent ensemble.

5. Dépôt et partage du code source (GitHub)

- Utiliser exclusivement le dépôt privé fourni pour l'équipe par l'enseignat, ne pas créer de dépôt personnel parallèle.
- Ajouter l'enseignant (et l'assistant) comme collaborateurs dès la création du dépôt, sans quoi le livrable L5 ne pourra pas être évalué.
- Ne jamais rendre le dépôt public, ne jamais inviter un membre d'une autre équipe.
- Committer régulièrement (l'historique des commits fait partie de l'évaluation des livrables L5 et L6).
- Le dépôt doit contenir, au minimum, à la date T6 : le code source complet et commenté, le fichier agent.manifest.json, le Dockerfile, un README expliquant comment lancer l'agent en local.

Si votre agent a besoin d'un type d'objet non répertorié dans INTER_AGENT_DATA_CONTRACT.doc, ouvrez une issue sur le dépôt public du noyau pour en demander l'ajout et ne l'inventez jamais unilatéralement.

6. Exigences techniques de conformité

- Le fichier manifeste agent.manifest.json, conforme au schéma imposé (avec les champs produces/consumes pour les agents métiers).
- Les endpoints obligatoires (/health, /tenant/activate, /tenant/deactivate, /tenant/{id}/status, /config-schema) et leur enveloppe de réponse standard (status, data, error).
- L'envoi des alertes/événements vers le webhook standardisé du noyau (cybersécurité), ou la publication/réception d'objets conformes au registre via /exchange/... (agent métier, endpoint /exchange/notify obligatoire si votre agent consomme des objets).
- L'isolation stricte des données par utilisateur/entreprise.
- Le packaging sous forme d'image Docker fonctionnelle et autonome.

Le non-respect du contrat d'intégration empêchera votre agent d'être branché au noyau lors de la phase finale d'assemblage, avec un impact direct et important sur la note du livrable L5 et sur la présentation finale (L8).

7. Livrables du projet

Identiques pour les deux filières dans leur structure ; seul le préfixe de nom de fichier change : utilisez CYBER pour les agent cybersécurité et MITIER pour la les agent automatisation métier (ex CYBER_DO_ScannerVuln.docx ou METIER_DO_AgentStock.docx).

Liv.	Contenu (centré sur VOTRE agent)	Volume / format	Nom de fichier
L1	Document d'orientation : agent choisi, objectif, justification, méthodologie, plan prévisionnel, sources (contrat d'intégration, registre des objets partagés pour la filière métier, outils open source), risques identifiés (dont, le cas échéant, les risques de dépendance vis-à-vis d'autres agents).	~1 page, Word, avant T2	<PREFIXE>_DO_NomAgent.docx
L2	Charte de projet : définition du projet-agent, objectifs et bénéfices, décomposition des livrables et des tâches, ressources, affectation des tâches, analyse des risques (techniques ET risque de non-conformité au contrat/registre).	6–7 pages, Arial 10/12, interligne 1,5, avant T3	<PREFIXE>_Charte_NomAgent.docx
L3	PV de la réunion de lancement de l'équipe agent (rôles, engagement collectif sur le contrat d'intégration).	1–2 pages, avant T4	<PREFIXE>_PVLancement_NomAgent.docx
L4	Planification : WBS/OBS/RACI, échéancier et jalons, dépendances (y compris dépendances fonctionnelles avec d'autres agents pour la filière métier), PERT, Gantt, estimation des coûts et efforts.	5–6 pages, avant T5	<PREFIXE>_PLAN_NomAgent.docx
L5	Rapport d'exécution : langage/frameworks/outils utilisés, versions développées (SemVer), problèmes rencontrés (dont conformité au contrat). Code source complet et commenté déposé sur le dépôt GitHub privé de l'équipe.	6–7 pages hors annexes, avant T6	<PREFIXE>_RAP_NomAgent.docx + dépôt GitHub
L6	Rapport de suivi : tableau de bord KPI (prévisionnel vs réalisé) ET état de conformité au contrat d'intégration (et au registre d'objets pour la filière métier).	3–5 pages hors annexes, avant T7	<PREFIXE>_Suivi_NomAgent.docx
L7	Bilan de projet : réalisations, écarts par rapport aux objectifs, perspectives, leçons apprises.	4–5 pages hors annexes, avant T8	<PREFIXE>_Bilan_NomAgent.docx
L8	Présentation orale (20 min + 5 min questions) avec DÉMONSTRATION LIVE de votre agent, activé et fonctionnel sur la plateforme du noyau.	PowerPoint uniquement, avant T9	<PREFIXE>_PRES_NomAgent.pptx

Présentation orale (L8) et critères généraux d'évaluation